

Integrantes: Sebastian Hernandez y Juan Esteban Ciro

CODIGO DE LOS ARCHIVOS

Pasajero.java

```
package aeropuerto;

public class Pasajero {

    private String nombre;
    private String numeroPasaporte;
    private String nacionalidad;

    public Pasajero(String nombre, String numeroPasaporte, String nacionalidad) {
        this.nombre = nombre;
        this.numeroPasaporte = numeroPasaporte;
        this.nacionalidad = nacionalidad;
    }

    // Getters
    public String getNombre() {
        return nombre;
    }

    public String getNumeroPasaporte() {
        return numeroPasaporte;
    }

    public String getNacionalidad() {
        return nacionalidad;
    }

    @Override
    public String toString() {
        return nombre + " (pasaporte " + numeroPasaporte + ", " + nacionalidad + ")";
    }
}
```

Vuelo.java

```
package aeropuerto;

import java.util.ArrayList;

public class Vuelo {

    private String identificador;
    private String ciudadOrigen;
    private String ciudadDestino;
    private double precio;
    private int numeroMaximoPasajeros;
    private ArrayList<Pasajero> pasajeros;

    public Vuelo(String identificador, String ciudadOrigen, String ciudadDestino, double
precio,
        int numeroMaximoPasajeros) {
        this.identificador = identificador;
        this.ciudadOrigen = ciudadOrigen;
        this.ciudadDestino = ciudadDestino;
        this.precio = precio;
        this.numeroMaximoPasajeros = numeroMaximoPasajeros;
        this.pasajeros = new ArrayList<>();
    }
}
```

```

// Getters
public String getIdentificador() {
    return identificador;
}

public String getCiudadOrigen() {
    return ciudadOrigen;
}

public String getCiudadDestino() {
    return ciudadDestino;
}

public double getPrecio() {
    return precio;
}

public int getNumeroMaximoPasajeros() {
    return numeroMaximoPasajeros;
}

public ArrayList<Pasajero> getPasajeros() {
    return pasajeros;
}

// true si se pudo reservar, false si ya estaba lleno. Quien llama decide qué imprimir
// con ese resultado – esta clase no imprime nada por su cuenta.
public boolean reservarAsiento(Pasajero pasajero) {
    if (pasajeros.size() >= numeroMaximoPasajeros) {
        return false;
    }
    pasajeros.add(pasajero);
    return true;
}

@Override
public String toString() {
    return identificador + ": " + ciudadOrigen + " -> " + ciudadDestino + " ($" + precio + ",
"
        + pasajeros.size() + "/" + numeroMaximoPasajeros + " puestos ocupados)";
}
}

```

Compania.java

```

package aeropuerto;

import java.util.ArrayList;

public class Compania {
    private String nombre;
    private ArrayList<Vuelo> vuelos;

    public Compania(String nombre) {
        this.nombre = nombre;
        this.vuelos = new ArrayList<>();
    }

    public String getNombre() {
        return nombre;
    }

    public ArrayList<Vuelo> getVuelos() {
        return vuelos;
    }
}

```

```

    }

    // agregar un vuelo a la compañía
    public void agregarVuelo(Vuelo vuelo) {
        vuelos.add(vuelo);
    }

    @Override
    public String toString() {
        return nombre + " - Vuelos: " + vuelos.size();
    }
}

```

Aeropuerto.java (padre)

```

package aeropuerto;

import java.util.ArrayList;

// Clase padre. Es "abstract" porque nunca vamos a crear un Aeropuerto genérico:
// todo aeropuerto real es público o privado. Esta clase solo guarda lo que
// ambos tipos tienen en común, y obliga a las hijas a definir el resto.
public abstract class Aeropuerto {
    private String nombre;
    private String ciudad;
    private String pais;
    private ArrayList<Compania> companias;

    public Aeropuerto(String nombre, String ciudad, String pais) {
        this.nombre = nombre;
        this.ciudad = ciudad;
        this.pais = pais;
        this.companias = new ArrayList<>();
    }

    public String getNombre() {
        return nombre;
    }

    public String getCiudad() {
        return ciudad;
    }

    public String getPais() {
        return pais;
    }

    public ArrayList<Compania> getCompanias() {
        return companias;
    }

    // agregar una compañía
    public void agregarCompania(Compania compania) {
        companias.add(compania);
    }

    @Override
    public String toString() {
        return nombre + " (" + ciudad + ", " + pais + ")" + " - Compañías: " + companias.size();
    }

    // Sin cuerpo a propósito: el padre no sabe si va a hablar de subvención o de
    // patrocinadores. Cada hija implementa esto con @Override, cada una a su manera.
    // Así el menú llama este único método sin preguntar antes qué tipo de aeropuerto es.
    public abstract String informacionDeFinanciacion();
}

```

```
}
```

AeropuertoPublico.java (hija)

```
package aeropuerto;

// Hereda de Aeropuerto: tiene todo lo del padre (nombre, ciudad, pais, companias)
// más el dato propio de este tipo, la subvención del gobierno.
public class AeropuertoPublico extends Aeropuerto {

    private double subvencionGobierno;

    public AeropuertoPublico(String nombre, String ciudad, String pais, double
subvencionGobierno) {
        // super(...) construye la parte "Aeropuerto" antes de construir la parte "pública".
        // Sin esta línea no compila: el padre no tiene un constructor vacío que llamar solo.
        super(nombre, ciudad, pais);
        this.subvencionGobierno = subvencionGobierno;
    }

    public double getSubvencionGobierno() {
        return subvencionGobierno;
    }

    // @Override confirma que esto reemplaza el método abstracto del padre, no que
    // crea uno nuevo por accidente (si el nombre no coincidiera exacto, el compilador
    avisaría).
    @Override
    public String informacionDeFinanciacion() {
        return "Aeropuerto público. Subvención del gobierno: $" + subvencionGobierno;
    }
}
```

AeropuertoPrivado.java (hija)

```
package aeropuerto;

import java.util.ArrayList;

// Igual que AeropuertoPublico, pero el dato propio es una lista, no un solo valor.
public class AeropuertoPrivado extends Aeropuerto {

    private ArrayList<String> empresasPatrocinadoras;

    public AeropuertoPrivado(String nombre, String ciudad, String pais) {
        super(nombre, ciudad, pais);
        this.empresasPatrocinadoras = new ArrayList<>();
    }

    public ArrayList<String> getEmpresasPatrocinadoras() {
        return empresasPatrocinadoras;
    }

    // Mismo patrón que agregarVuelo() en Compania y agregarCompania() en Aeropuerto:
    // la lista empieza vacía y se llena de a una empresa a la vez.
    public void agregarPatrocinador(String empresa) {
        empresasPatrocinadoras.add(empresa);
    }

    @Override
    public String informacionDeFinanciacion() {
        return "Aeropuerto privado. Empresas patrocinadoras: " + empresasPatrocinadoras;
    }
}
```

```
}
```

Utilidades.java

```
package aeropuerto;

import java.util.ArrayList;

/*
 * Clase de utilidades generales para el programa. Solo guarda metodos recurrentes que se
usan en
 * varias partes del programa, como imprimir el encabezado con los nombres de los integrantes
del
 * equipo.
 */

public class Utilidades {

    // Imprime el encabezado con los nombres de los dos integrantes del equipo.
    public static void imprimirEncabezado() {
        System.out.println("=====");
        System.out.println("Sistema de Gestion Aeroportuaria");
        System.out.println("Integrantes: Sebastian Hernandez y Juan Esteban Ciro");
        System.out.println("=====");
    }

    // Limpia la consola. El comando real depende del sistema operativo (cls en Windows,
    // clear en Linux/Mac), por eso se detecta con os.name antes de decidir cuál correr.
    // Si por algún motivo falla (ej. corriendo dentro de un IDE que no lo soporta bien),
    // no truena el programa, simplemente no se limpia.
    public static void limpiarConsola() {
        try {
            if (System.getProperty("os.name").toLowerCase().contains("windows")) {
                new ProcessBuilder("cmd", "/c", "cls").inheritIO().start().waitFor();
            } else {
                new ProcessBuilder("clear").inheritIO().start().waitFor();
            }
        } catch (Exception e) {
            // No es crítico si no se pudo limpiar, el programa sigue funcionando igual.
        }
    }

    // Atajo para no repetir estas dos líneas en cada opción del menú.
    public static void cambiarPagina() {
        limpiarConsola();
        imprimirEncabezado();
    }

    // arma una lista con los datos minimos solicitados de aeropuertos con sus respectivas
compañías,
    // vuelos y pasajeros para usarla en el programa
    public static ArrayList<Aeropuerto> cargarDatos() {
        // --- Pasajeros (5) ---
        Pasajero p1 = new Pasajero("Sebastian Hernandez", "AB123456", "Colombia");
        Pasajero p2 = new Pasajero("Juan Esteban Ciro", "CD987654", "Colombia");
        Pasajero p3 = new Pasajero("Simón Camilo", "EF112233", "Venezuela");
        Pasajero p4 = new Pasajero("Santiago Gomez", "GH445566", "Colombia");
        Pasajero p5 = new Pasajero("Cristobal Toro", "IJ778899", "Canada");

        // --- Vuelos (6), repartidos entre las 3 compañías ---
        Vuelo av205 = new Vuelo("AV205", "Medellin", "Bogota", 250000.0, 2);
        av205.reservarAsiento(p1);
        av205.reservarAsiento(p2);

        Vuelo av310 = new Vuelo("AV310", "Bogota", "Cartagena", 320000.0, 4);
    }
}
```

```

Vuelo la100 = new Vuelo("LA100", "Bogota", "Cali", 210000.0, 3);
la100.reservarAsiento(p3);
la100.reservarAsiento(p4);

Vuelo la150 = new Vuelo("LA150", "Bogota", "Miami", 950000.0, 3);

Vuelo p5200 = new Vuelo("P5200", "Barranquilla", "Bogota", 180000.0, 5);
p5200.reservarAsiento(p5);

Vuelo p5210 = new Vuelo("P5210", "Barranquilla", "Cartagena", 150000.0, 2);

// --- Compañías (3) ---
Compania avianca = new Compania("Avianca");
avianca.agregarVuelo(av205);
avianca.agregarVuelo(av310);

Compania latam = new Compania("Latam");
latam.agregarVuelo(la100);
latam.agregarVuelo(la150);

Compania wingo = new Compania("Wingo");
wingo.agregarVuelo(p5200);
wingo.agregarVuelo(p5210);

// --- Aeropuertos (3: 2 públicos, 1 privado) ---
Aeropuerto jmc = new AeropuertoPublico("Jose Maria Cordova", "Rionegro", "Colombia",
8000000.0);
jmc.agregarCompania(avianca);

// Avianca también opera en El Dorado: la misma compañía, el mismo objeto, agregado a dos
// aeropuertos distintos
Aeropuerto elDorado = new AeropuertoPrivado("El Dorado Internacional", "Bogota",
"Colombia");
elDorado.agregarCompania(avianca);
elDorado.agregarCompania(latam);
// Cast obligatorio: elDorado es de tipo Aeropuerto, y agregarPatrocinador() solo
// existe en AeropuertoPrivado, no en el padre.
((AeropuertoPrivado) elDorado).agregarPatrocinador("Grupo Bolivar");
((AeropuertoPrivado) elDorado).agregarPatrocinador("Corficolombiana");

Aeropuerto cortissoz =
    new AeropuertoPublico("Ernesto Cortissoz", "Barranquilla", "Colombia", 3000000.0);
cortissoz.agregarCompania(wingo);

ArrayList<Aeropuerto> aeropuertos = new ArrayList<>();
aeropuertos.add(jmc);
aeropuertos.add(elDorado);
aeropuertos.add(cortissoz);

return aeropuertos;
}
}

```

Principal.java

```
package aeropuerto;

import java.util.ArrayList;
import java.util.Scanner;

public class Principal {

    // La lista que arma Utilidades.cargarDatos() se guarda aquí una sola vez,
    // al arrancar, para que todas las opciones del switch la puedan usar.
    // static porque main() es static y nunca creamos un new Principal(); todo vive
    // a nivel de la clase, no de un objeto.
    private static ArrayList<Aeropuerto> aeropuertos;
    private static Scanner sc = new Scanner(System.in);

    public static void main(String[] args) {
        aeropuertos = Utilidades.cargarDatos();
        String op; // String, no int: así una letra no revienta el programa, solo cae al default
        while (true) {
            Utilidades.cambiarPagina(); // pantalla limpia + encabezado antes de mostrar el menú
            mostrarMenu();
            op = sc.nextLine().trim();
            Utilidades.cambiarPagina(); // se limpia otra vez para que el resultado no quede pegado
al
                                // menú

            switch (op) {
                case "1":
                    imprimirTitulo("AEROPUERTOS GESTIONADOS");
                    consultarAeropuertos();
                    break;
                case "2":
                    imprimirTitulo("FINANCIACIÓN DE UN AEROPUERTO");
                    verFinanciacion();
                    break;
                case "3":
                    imprimirTitulo("COMPAÑÍAS DE UN AEROPUERTO");
                    verCompaniasDeAeropuerto();
                    break;
                case "4":
                    imprimirTitulo("VUELOS DE UNA COMPAÑÍA");
                    verVuelosDeCompania();
                    break;
                case "5":
                    imprimirTitulo("VUELOS ENTRE DOS CIUDADES");
                    buscarVuelosEntreCiudades();
                    break;
                case "0":
                    System.out.println("Saliendo...");
                    sc.close(); // solo aquí, una vez: cerrarlo antes cortaría la lectura en la
siguiente
                                // vuelta

                    return;
                default:
                    System.out.println("=====");
                    System.out.println("Opción no válida. Intente de nuevo.");
                    System.out.println("=====");
            }

            System.out.println();
            System.out.println("Presione Enter para continuar...");
            sc.nextLine();
        }
    }
}
```

```

public static void mostrarMenu() {
    System.out.println("1. Consultar Aeropuertos");
    System.out.println("2. Financiación de Aeropuertos");
    System.out.println("3. Compañías de los Aeropuertos");
    System.out.println("4. Vuelos de la Compañía");
    System.out.println("5. Buscar Vuelos");
    System.out.println("0. Salir");
    System.out.print("Ingrese una opción: ");
}

private static void imprimirTitulo(String titulo) {
    System.out.println();
    System.out.println(">>> " + titulo + " <<<");
    System.out.println("-----");
}

// -----
// Opción 1: públicos y privados por separado. Nombre, ciudad y país.
// -----
private static void consultarAeropuertos() {
    // instanceof aquí SÍ está bien: solo separa en dos grupos para imprimir, no decide
    // comportamiento distinto (eso es lo que hace mal usar instanceof). Comparar con
    // verFinanciacion() de abajo, que es el caso real de polimorfismo.
    System.out.println("--- Aeropuertos públicos ---");
    for (Aeropuerto a : aeropuertos) {
        if (a instanceof AeropuertoPublico) {
            System.out.println(a); // usa Aeropuerto.toString()
        }
    }

    System.out.println("--- Aeropuertos privados ---");
    for (Aeropuerto a : aeropuertos) {
        if (a instanceof AeropuertoPrivado) {
            System.out.println(a); // usa Aeropuerto.toString()
        }
    }
}

// -----
// Opción 2: LA CLAVE DEL PROYECTO. Sin instanceof: el método abstracto
// informacionDeFinanciacion() responde distinto según la subclase real.
// -----
private static void verFinanciacion() {
    System.out.print("Nombre del aeropuerto (nombre completo e igual): ");
    String nombre = sc.nextLine();

    Aeropuerto encontrado = buscarAeropuertoPorNombre(nombre); // busca en la lista
    if (encontrado == null) {
        System.out.println("No se encontró un aeropuerto con ese nombre.");
        return; // corta aquí, no sigue
    }

    System.out.println(encontrado.informacionDeFinanciacion()); // cada subclase responde
    distinto
}

// -----
// Opción 3: compañías que operan en un aeropuerto.
// -----
private static void verCompaniasDeAeropuerto() {
    System.out.print("Nombre del aeropuerto (nombre completo e igual): ");
    String nombre = sc.nextLine();

    Aeropuerto encontrado = buscarAeropuertoPorNombre(nombre); // paso 1: buscar el
    aeropuerto

```

```

    if (encontrado == null) {
        System.out.println("No se encontró un aeropuerto con ese nombre.");
        return;
    }

    if (encontrado.getCompanias().isEmpty()) { // aeropuerto existe pero sin companias
        System.out.println("Este aeropuerto no tiene compañías registradas.");
        return;
    }

    for (Compania c : encontrado.getCompanias()) { // paso 2: recorrer sus companias
        System.out.println(c); // usa Compania.toString()
    }
}

// -----
// Opción 4: vuelos de una compañía puntual en un aeropuerto puntual.
// -----
private static void verVuelosDeCompania() {
    System.out.print("Nombre del aeropuerto (nombre completo e igual): ");
    String nombreAeropuerto = sc.nextLine();

    Aeropuerto encontrado = buscarAeropuertoPorNombre(nombreAeropuerto); // paso 1: el
aeropuerto
    if (encontrado == null) {
        System.out.println("No se encontró un aeropuerto con ese nombre.");
        return;
    }

    System.out.print("Nombre de la compañía: ");
    String nombreCompania = sc.nextLine();

    Compania encontrada = null;
    for (Compania c : encontrado.getCompanias()) { // paso 2: buscar SOLO entre las companias
de ESE
                                                // aeropuerto
        if (c.getNombre().equalsIgnoreCase(nombreCompania)) {
            encontrada = c;
            break; // ya la encontró, no sigue buscando
        }
    }

    if (encontrada == null) { // existe la compania, pero no en este aeropuerto
        System.out.println("Esa compañía no opera en ese aeropuerto.");
        return;
    }

    if (encontrada.getVuelos().isEmpty()) {
        System.out.println("Esa compañía no tiene vuelos registrados.");
        return;
    }

    for (Vuelo v : encontrada.getVuelos()) { // paso 3: sus vuelos
        System.out.println(v); // usa Vuelo.toString()
    }
}

// -----
// Opción 5: buscar por ruta en TODOS los aeropuertos, sin repetir un
// mismo vuelo si su compañía opera en más de un aeropuerto (Avianca en
// este caso, ver Utilidades.cargarDatos()).
// -----
private static void buscarVuelosEntreCiudades() {
    System.out.print("Ciudad de origen: ");
    String origen = sc.nextLine();

```

```

System.out.print("Ciudad de destino: ");
String destino = sc.nextLine();

// yaMostrados.contains(v) funciona sin que Vuelo tenga equals() propio, porque son
// literalmente el mismo objeto en memoria (Avianca comparte sus vuelos entre dos
// aeropuertos, ver Utilidades.cargarDatos()) – no dos copias con los mismos datos.
ArrayList<Vuelo> yaMostrados = new ArrayList<>(); // para no repetir el mismo vuelo
boolean encontroAlguno = false;

for (Aeropuerto a : aeropuertos) { // recorre cada aeropuerto...
    for (Compania c : a.getCompanias()) { // ...cada compania de ese aeropuerto...
        for (Vuelo v : c.getVuelos()) { // ...cada vuelo de esa compania
            boolean coincideRuta = v.getCiudadOrigen().equalsIgnoreCase(origen)
                && v.getCiudadDestino().equalsIgnoreCase(destino);

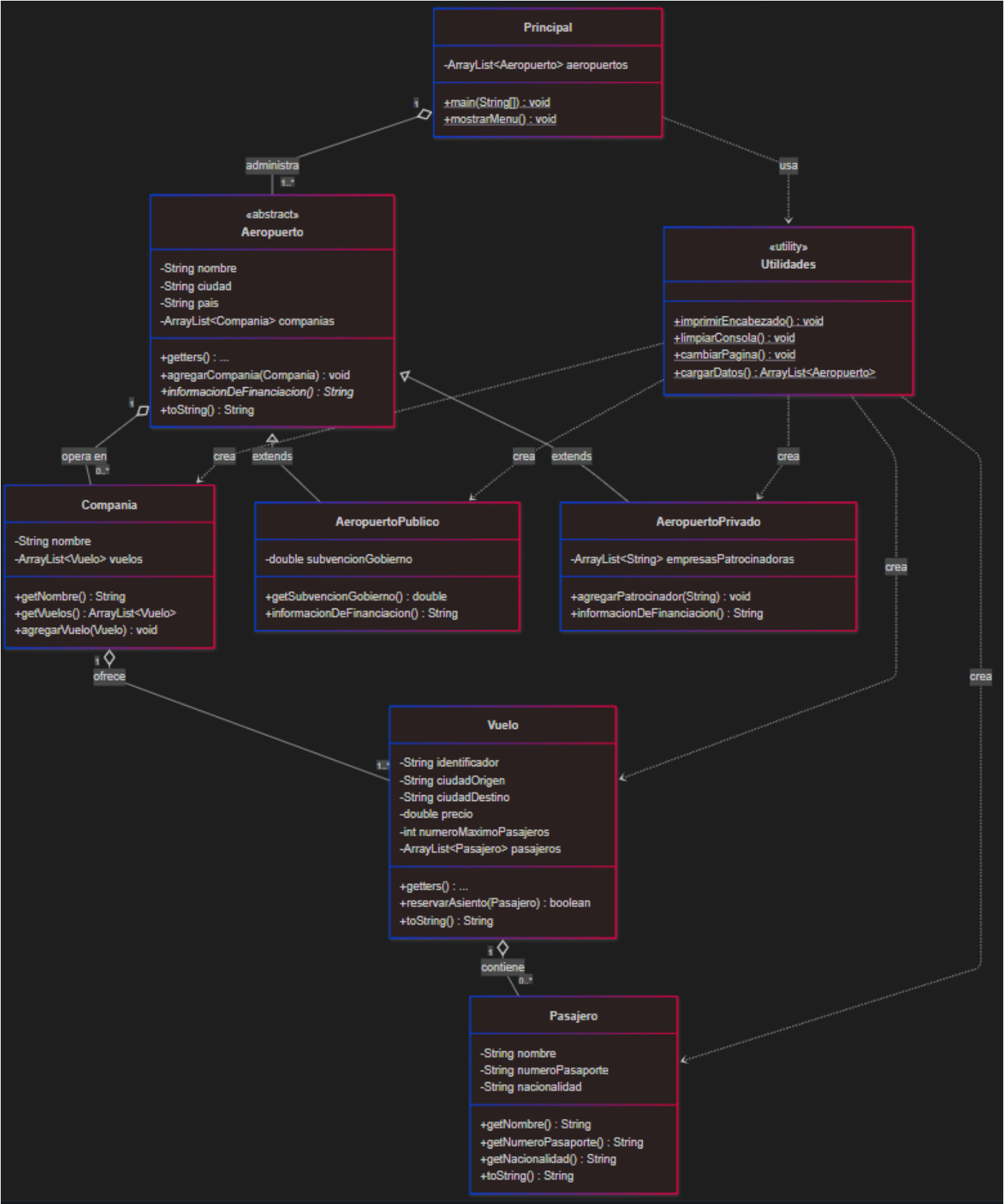
            if (coincideRuta && !yaMostrados.contains(v)) { // coincide y no se mostró antes
                System.out.println(v); // usa Vuelo.toString()
                yaMostrados.add(v); // marcarlo como ya mostrado
                encontroAlguno = true;
            }
        }
    }
}

if (!encontroAlguno) { // ningun vuelo cumpliero la condicion
    System.out.println("No se encontraron vuelos para esa ruta.");
}

// Método de apoyo reutilizado por las opciones 2, 3 y 4.
private static Aeropuerto buscarAeropuertoPorNombre(String nombre) {
    for (Aeropuerto a : aeropuertos) {
        if (a.getNombre().equalsIgnoreCase(nombre)) {
            return a;
        }
    }
    return null;
}
}

```

DIAGRAMA DE FLUJO (archivo tipo .mermaid)



%% =====
%% GUÍA DE VISUALIZACIÓN:
%% Para ver este diagrama de clases en VS Code (extensión Mermaid Preview):
%% 1. Presiona: Ctrl + Shift + P (o Cmd + Shift + P en macOS)
%% 2. Escribe y selecciona: Mermaid Preview: Preview Diagram
%% =====

```
classDiagram
class Pasajero {
    -String nombre
    -String numeroPasaporte
    -String nacionalidad
    +getNombre() String
    +getNumeroPasaporte() String
    +getNacionalidad() String
    +toString() String
}

class Vuelo {
```

```

    -String identificador
    -String ciudadOrigen
    -String ciudadDestino
    -double precio
    -int numeroMaximoPasajeros
    -ArrayList~Pasajero~ pasajeros
    +getters()...
    +reservarAsiento(Pasajero) boolean
    +toString() String
}

class Compania {
    -String nombre
    -ArrayList~Vuelo~ vuelos
    +getNombre() String
    +getVuelos() ArrayList~Vuelo~
    +agregarVuelo(Vuelo) void
}

class Aeropuerto {
    <<abstract>>
    -String nombre
    -String ciudad
    -String pais
    -ArrayList~Compania~ companias
    +getters()...
    +agregarCompania(Compania) void
    +informacionDeFinanciacion() String*
    +toString() String
}

class AeropuertoPublico {
    -double subvencionGobierno
    +getSubvencionGobierno() double
    +informacionDeFinanciacion() String
}

class AeropuertoPrivado {
    -ArrayList~String~ empresasPatrocinadoras
    +agregarPatrocinador(String) void
    +informacionDeFinanciacion() String
}

class Principal {
    -ArrayList~Aeropuerto~ aeropuertos
    +main(String[])$ void
    +mostrarMenu()$ void
}

class Utilidades {
    <<utility>>
    +imprimirEncabezado()$ void
    +limpiarConsola()$ void
    +cambiarPagina()$ void
    +cargarDatos()$ ArrayList~Aeropuerto~
}

Vuelo "1" o-- "0..*" Pasajero : contiene
Compania "1" o-- "1..*" Vuelo : ofrece
Aeropuerto "1" o-- "0..*" Compania : opera en
Principal "1" o-- "1..*" Aeropuerto : administra

Aeropuerto <|-- AeropuertoPublico : extends
Aeropuerto <|-- AeropuertoPrivado : extends

```

```
Principal ..> Utilidades : usa
Utilidades ..> AeropuertoPublico : crea
Utilidades ..> AeropuertoPrivado : crea
Utilidades ..> Compania : crea
Utilidades ..> Vuelo : crea
Utilidades ..> Pasajero : crea
```

PRUEBAS DE FUNCIONAMIENTO CORRECTO

```
=====
Sistema de Gestion Aeroportuaria
Integrantes: Sebastian Hernandez y Juan Esteban Ciro
=====
1. Consultar Aeropuertos
2. Financiación de Aeropuertos
3. Compañías de los Aeropuertos
4. Vuelos de la Compañía
5. Buscar Vuelos
0. Salir
Ingrese una opción:
```

```
=====
Sistema de Gestion Aeroportuaria
Integrantes: Sebastian Hernandez y Juan Esteban Ciro
=====
1. Consultar Aeropuertos
2. Financiación de Aeropuertos
3. Compañías de los Aeropuertos
4. Vuelos de la Compañía
5. Buscar Vuelos
0. Salir
Ingrese una opción:
```

```
=====
Sistema de Gestion Aeroportuaria
Integrantes: Sebastian Hernandez y Juan Esteban Ciro
=====

>>> FINANCIACIÓN DE UN AEROPUERTO <<<
-----
Nombre del aeropuerto (nombre completo e igual): el dorado internacional
Aeropuerto privado. Empresas patrocinadoras: [Grupo Bolivar, Corficolombiana]

Presione Enter para continuar...
█
```

```
=====
Sistema de Gestion Aeroportuaria
Integrantes: Sebastian Hernandez y Juan Esteban Ciro
=====

>>> COMPAÑÍAS DE UN AEROPUERTO <<<
-----
Nombre del aeropuerto (nombre completo e igual): el dorado internacional
Avianca - Vuelos: 2
Latam - Vuelos: 2

Presione Enter para continuar...
█
```

```
=====
Sistema de Gestion Aeroportuaria
Integrantes: Sebastian Hernandez y Juan Esteban Ciro
=====

>>> COMPAÑÍAS DE UN AEROPUERTO <<<
-----
Nombre del aeropuerto (nombre completo e igual): el dorado internacional
Avianca - Vuelos: 2
Latam - Vuelos: 2

Presione Enter para continuar...
█
```

```
=====
Sistema de Gestion Aeroportuaria
Integrantes: Sebastian Hernandez y Juan Esteban Ciro
=====
```

```
>>> VUELOS DE UNA COMPAÑÍA <<<
```

```
-----
Nombre del aeropuerto (nombre completo e igual): el dorado internacional
```

```
Nombre de la compañía: avianca
```

```
AV205: Medellin -> Bogota ($250000.0, 2/2 puestos ocupados)
```

```
AV310: Bogota -> Cartagena ($320000.0, 0/4 puestos ocupados)
```

```
Presione Enter para continuar...
```

```
█
```

```
=====
Sistema de Gestion Aeroportuaria
Integrantes: Sebastian Hernandez y Juan Esteban Ciro
=====
```

```
>>> VUELOS ENTRE DOS CIUDADES <<<
```

```
-----
Ciudad de origen: bogota
```

```
Ciudad de destino: miami
```

```
LA150: Bogota -> Miami ($950000.0, 0/3 puestos ocupados)
```

```
Presione Enter para continuar...
```

```
█
```

ERRORES DEL USUARIO

```
=====
Sistema de Gestion Aeroportuaria
Integrantes: Sebastian Hernandez y Juan Esteban Ciro
=====

>>> FINANCIACIÓN DE UN AEROPUERTO <<<
-----
Nombre del aeropuerto (nombre completo e igual): Chancha Volador
No se encontró un aeropuerto con ese nombre.

Presione Enter para continuar...
█
```

```
=====
Sistema de Gestion Aeroportuaria
Integrantes: Sebastian Hernandez y Juan Esteban Ciro
=====

>>> COMPAÑÍAS DE UN AEROPUERTO <<<
-----
Nombre del aeropuerto (nombre completo e igual): Has
No se encontró un aeropuerto con ese nombre.

Presione Enter para continuar...
█
```

```
=====
Sistema de Gestion Aeroportuaria
Integrantes: Sebastian Hernandez y Juan Esteban Ciro
=====

>>> VUELOS DE UNA COMPAÑÍA <<<
-----
Nombre del aeropuerto (nombre completo e igual): Ernesto Cortissoz
Nombre de la compañía: Comfama
Esa compañía no opera en ese aeropuerto.

Presione Enter para continuar...
█
```

```
=====
Sistema de Gestion Aeroportuaria
Integrantes: Sebastian Hernandez y Juan Esteban Ciro
=====

>>> VUELOS ENTRE DOS CIUDADES <<<
-----
Ciudad de origen: El Peñol
Ciudad de destino: Marinilla
No se encontraron vuelos para esa ruta.

Presione Enter para continuar...
█
```